

Advance Java Technologies

Unit – 1 :

1. What is java servlet API ? explain in detail ?

The **Java Servlet API** is a set of **classes and interfaces** provided by Java to create and manage **servlets**.

A servlet is a Java program that runs on the server and handles client requests, mainly **HTTP requests**, and sends responses back to the client. The Servlet API gives the predefined tools needed to do this.

Java Servlet API is a collection of packages, interfaces, and classes used to develop server-side web components that process client requests and generate dynamic responses.

Main purpose of Servlet API :

- receive client requests
- process data on the server
- generate dynamic responses
- manage servlet life cycle
- handle sessions and cookies

The **Java Servlet API** consists of two main packages:

1. javax.servlet Package :

Interfaces

- **Servlet** – Defines lifecycle methods (init(), service(), destroy())
- **ServletRequest** – Represents client request data
- **ServletResponse** – Represents response sent to client
- **ServletConfig** – Provides servlet configuration parameters

- **ServletContext** – Represents the whole web application environment
- **RequestDispatcher** – Used to forward/include requests

Classes

- **GenericServlet** – Abstract class implementing Servlet (protocol-independent)
- **ServletInputStream** – Reads binary data from request
- **ServletOutputStream** – Writes binary data to response

2. javax.servlet.http Package :

Interfaces

- **HttpServletRequest** – Handles HTTP request data (headers, cookies, parameters)
- **HttpServletResponse** – Handles HTTP response (status, cookies, redirect)
- **HttpSession** – Manages user session across multiple requests
- **HttpSessionListener** – Tracks session creation/destruction

Classes

- **HttpServlet** – Abstract class for HTTP-based servlets
- **Cookie** – Stores small data on client side
- **HttpServletResponseWrapper** – Wraps response object
- **HttpServletRequestWrapper** – Wraps request object

Q. What are the various steps in servlet life cycle goes through illustrate with diagram ?

The **Servlet Life Cycle** defines the stages through which a servlet passes from its creation to destruction. It is managed by the **servlet container**.

The servlet uses several important methods defined in the packages to manage the life cycle and handle client request.

1. The **init()** method :

The `init()` method is called only once when the servlet is first loaded into memory. It is used for initializing task like opening database connection, reading configuring file, etc.

Syntax :

```
Public void init(){  
}
```

2. **Service()** Method :

The `service()` method is called each time a request is made to the servlet. It determines the type of request and then calls the appropriate method.

Syntax :

```
Public void service( ServletRequest req, ServletResponse res)
```

Throws `ServletException`, `IOException`

```
{  
}
```

3. **doGet()** Method :

The doGet() method is called by the service() when the http request type is GET. It is used to handle request like fetching data or displaying web pages.

Syntax :

```
Public void doGet( HttpServletRequest req, HttpServletResponse res)
```

Throws servletException, IOException

```
{  
}
```

4. doPost() Method :

It is called by the service() method when the Http request type is POST. It is used to handle form submission and data sent from the client to the server.

Syntax :

```
Public void doPost( HttpServletRequest req, HttpServletResponse res)
```

Throws servletException, IOException

```
{  
}
```

5. destroy() Method :

The destroy() Method is called once when the servlet is about to be removed from memory. Used to release resources such as closing file, network connections, or database connection.

Syntax :

```
Public void destroy(){
```

```
}
```

Q. what is JDBC , explain it's architecture ?

The JDBC stands for java database connectivity.

JDBC driver is a software component that enables java application to interact with the database.

It is a api in java that allows java application to interact with different database.

It provides a method to connect, execute queries and retrieve result.

Uses of JDBC

- Connect Java program to database
- Execute SQL queries
- Retrieve and update data
- Manage database transactions

JDBC Architecture :

The JDBC architecture defines how a Java application communicates with a database. It mainly consists of two layers:

1. Java Application :

- The user program written in Java.
- Uses JDBC API to interact with the database.

Eg : A login system or banking application.

JDBC API

- A set of interfaces and classes provided in java.sql package.
- Used to send SQL queries and process results.

DriverManager

- A class that manages database drivers.
- Establishes connection between Java application and database.

Role:

- Loads drivers
- Selects appropriate driver
- Creates connection object

Q. Explain the JDBC driver types ?

JDBC drivers are software components that enable Java applications to communicate with databases. There are **four types of JDBC drivers**, classified based on how they interact with the database.

1. **Type 1** – JDBC-ODBC Bridge Driver :

The Type-1 driver uses ODBC driver to connect to the database.

It converts jdbc method into ODBC calls

It also called universal driver

It can be used to connect any of database.

Advan :

- Easy to use and implement
- Can connect to any database supporting ODBC

DisAdvan :

- Slow performance due to multiple conversions
- Requires ODBC installation

2. **Type-2** Native API Driver :

The native API uses client side libraries of database.

It convert JDBC method calls into native call of the database API.

It required local API to interact with different database.

Data transfer is more secure as compared to type-1.

Advan :

- Better performance than Type 1
- Efficient for specific databases

Dis :

- Requires native libraries installation
- Platform dependent

3. Type-3 Network Protocol Driver :

It uses middleware to convert JDBC calls into database-specific protocols.

It can convert to multiple database using one driver.

Advan :

- Platform independent
- Supports multiple databases

Dis :

- Requires middleware server
- Slower due to extra network layer

4. Type-4 Thin Driver :

It is written completely in java.

It interact directly with database.

It does not required any native database library that why it's known as thin driver.

Advan :

- Fastest performance
- Platform independent
- No need for external libraries

Dis:

- Database-specific driver required

UNIT- 2

Q. What is Request Dispatcher Interface ?

The Request dispatcher is an interface in java servlet that allows you to forward a request from one servlet to another resource, it may be html, servlet or jsp.

This interface also be used to include the content of another resource, it is one of the way of servlet collaboration.

There are two methods defined in the request dispatcher interface.

1. Forward()

It transfer the request from one servlet to another resource within the same server.

The control is completely transferred and the original servlet cannot write to the response after forwarding.

Syntax :

```
RequestDispatcher rd = request.getRequestDispatcher("demo.jsp");  
rd .forward(request, response);
```

2. Include():

It includes the content of another resource in the current response.

The control returns back to the calling servlet after inclusion.

Both the resources can contribute to the same response.

Syntax:

```
RequestDispatcher rd = request.getRequestDispatcher("demo.jsp");  
rd.include(request, response);
```

Q. What is cookies ? Discuss their method , Advantages and Disadvantages :

A **Cookie** is a small piece of data stored on the client's browser by the server. It is used to maintain user-specific information such as login details, preferences, and session tracking.

It used to remember information about the user between multiple http request, since http is a stateless protocol, so cookies help in state management , meaning they maintain user-specific data across multiple pages or visits.

Method of cookies class :

◆ 1. **setMaxAge(int expiry)**

- Sets the maximum time (in seconds) for which the cookie will be stored in the browser.
- If set to 0, the cookie is deleted immediately; if set to -1, it is stored until browser closes.

◆ 2. **getName()**

- Returns the name of the cookie.
- Used to identify a specific cookie among multiple cookies.

◆ 3. **getValue()**

- Returns the value stored in the cookie.
 - Helps in retrieving user-related information.
-

◆ 4. **setValue(String value)**

- Sets or updates the value of the cookie.
 - Useful when modifying user data stored in cookie.
-

◆ 5. **getMaxAge()**

- Returns the maximum age of the cookie.
 - Helps to check how long the cookie will remain valid.
-

◆ 6. **setPath(String path)**

- Specifies the path for which the cookie is valid.
 - Ensures that the cookie is sent only for specific resources.
-

◆ 7. **getPath()**

- Returns the path of the cookie.
 - Used to know where the cookie is accessible.
-

◆ **Advantages of Cookies**

- **Maintains User Session:**

Cookies help in maintaining user sessions across multiple requests, such as keeping a user logged in.

- **Improves User Experience:**

Stores user preferences like language, theme, or settings, making websites more personalized.

- **Reduces Server Load:**

Since data is stored on the client side, it reduces the need to store all session data on the server.

- **Simple and Easy to Use:**

Cookies are easy to implement and widely supported by all browsers.

◆ Disadvantages of Cookies

- **Security Risks:**

Cookies can be intercepted or modified by attackers, leading to data theft (e.g., session hijacking).

- **Limited Storage Size:**

Cookies can store only a small amount of data (usually around 4KB).

- **Client-Side Storage:**

Data is stored on the user's machine, so it can be deleted or disabled by the user.

- **Performance Overhead:**

Cookies are sent with every request, which can slightly increase network traffic.

Q. How the cookies can be **set** and **retrieve** using servlet ?

Cookies in servlets are used to **store and retrieve user information** on the client side. They are created by the server and stored in the browser, and then sent back to the server with each request.

Setting cookies :

```
Cookie cookie = new Cookie("username", "Krish");
```

Cookies are created by the server using the servlet and sent to the client's browser along with the response.

While setting a cookie, the server specifies:

the **name and value** of the cookie

Once the response is sent, the browser **stores the cookie locally** on the user's system.

Retrieving Cookies :

When the client sends a request to the server again, the browser automatically **attaches all stored cookies** related to that website.

The servlet retrieves these cookies from the request and:

- identifies the required cookie using its **name**
- extracts its **value**

Q. What is session tracking and explain it's method :

A **session** is a way to maintain state between multiple requests of a user during interaction with a web application.

Session tracking is a technique used in web application to maintain a user's state and data across multiple request.

Since http is a stateless protocol, each request from user is independent.

The server doesn't automatically remember previous interaction

Session tracking helps the server identify the same user across multiple request and store temporary data like login info, cards items, etc.

Methods of Session Tracking :

1. Cookies

- Data is stored on the **client side (browser)**
- Server sends cookies and browser stores them
- Cookies are sent back with each request

2. URL Rewriting

- Session data is appended to the URL
- Data is passed as query parameters

3. Hidden Form Fields

- Data is stored in **hidden fields of HTML forms**
- Passed when form is submitted

4. HttpSession (Server-side Session)

- Data is stored on the **server side**
- Each user gets a unique session ID

Q. Explain what is httpsession along with it's lifecycle and various method

HttpSession is used to store and maintain user-specific data on the server across multiple requests in a web application.

1. **setAttribute(String name, Object value)**

- Stores data in the session
- Used to save user information

◆ 2. **getAttribute(String name)**

- Retrieves stored session data
- Used to access user information

◆ **3. removeAttribute(String name)**

- Removes a specific attribute from session
- Used when data is no longer needed

◆ **4. invalidate()**

- Destroys the entire session
- Used during logout

◆ **5. getId()**

- Returns unique session ID
- Used to identify user session

◆ **6. getCreationTime()**

- Returns the time when session was created

◆ **7. getLastAccessedTime()**

- Returns last time user accessed session

◆ **8. setMaxInactiveInterval(int interval)**

- Sets session timeout period
- Session expires after inactivity

◆ **HttpSession Lifecycle**

◆ **1. Creation**

- Session is created when user sends first request

- Server assigns a unique **session ID**

◆ 2. Usage

- Session is used to store and retrieve data
- Continues as long as user is active

◆ 3. Inactive Period

- If user is inactive for a specific time, session becomes idle

◆ 4. Destruction

- Session is destroyed when:
 - User logs out (invalidate())
 - Session timeout occurs
 - Server shuts down

Q. How does the non-blocking I/O works in servlets ?

Non-blocking I/O in servlets is a mechanism where the server can handle input/output operations **without waiting** for them to complete, allowing it to process multiple requests efficiently.

Need for Non-Blocking I/O

- In traditional (blocking) I/O, a thread **waits** until the operation (read/write) is completed
- This causes **thread blocking** and reduces performance under heavy load
 - 👉 Non-blocking I/O solves this by allowing threads to handle **other tasks while waiting**
- **Client sends request**
 - Request may contain large data (file upload, streaming, etc.)

- **Servlet starts processing asynchronously**
 - Instead of waiting, the servlet uses non-blocking mode
- **I/O operation begins**
 - Data is read/written in parts (chunks)
- **Listener monitors readiness**
 - The servlet registers listeners (like ReadListener / WriteListener)
 - These listeners get notified when data is available
- **Processing continues without blocking thread**
 - The server thread is free to handle other requests
- **Response is completed when data processing finishes**

Q. How file can be uploaded in java Servlet. What are the steps and api involved :

Steps to Upload File in Servlet

◆ 1. Create HTML Form

- Use a form with:
 - method = "post"
 - enctype = "multipart/form-data"
- 👉 This encoding is required to send file data

◆ 2. Create Servlet

- Create a servlet to handle the request
- Map the servlet using annotations or web.xml

◆ 3. Enable Multipart Configuration

- Configure servlet to support file upload

- This allows handling of file data sent in request
-

◆ 4. Receive the Request

- Servlet receives the request containing file data
 - The request is processed as multipart content
-

◆ 5. Extract File Data

- Retrieve file from request using API
 - Access file name, size, and content
-

◆ 6. Save the File

- Store the file on server (in a directory)
 - Can also store file path in database
-

◆ 7. Send Response

- After upload, send confirmation message to user

UNIT - 3

Q. Discuss the various steps the jsp lifecycle goes through :

The **JSP (JavaServer Pages) Life Cycle** describes the stages a JSP page goes through from its creation to destruction. It is managed by the **JSP container (web container)**.

Steps in JSP Life Cycle

Translation Phase

- The JSP page is **converted into a servlet (.java file)** by the container.
- This happens only once when the JSP is accessed for the first time or modified.

👉 JSP → Servlet (Java file)

◆ 2. Compilation Phase

- The generated servlet file is **compiled into a class file (.class)**.
- If there are errors in JSP, they appear in this phase.

👉 .java → .class

◆ 3. Class Loading

- The compiled servlet class is **loaded into memory** by the class loader.
 - Prepares the servlet for execution.
-

◆ 4. Instantiation

- The container creates an **object (instance)** of the servlet class.
 - This happens only once.
-

◆ 5. Initialization – `jspInit()`

- The container calls **`jspInit()` method**.
 - Executed only once after object creation.
 - Used to initialize resources (database connection, etc.)
-

◆ 6. Request Processing – `_jspService()`

- For every client request, **`_jspService()` method** is called.
 - Generates dynamic response (HTML content).
 - This is the most important phase.
-

◆ 7. Destruction – `jspDestroy()`

- The container calls **`jspDestroy()` method** when JSP is removed.
- Used to release resources.
- Called only once.

Q. State the Difference between JSP and Servlet ?

- A **Servlet** is a Java class used for handling business logic and processing client requests, whereas a **JSP (Java Server Pages)** is used for designing the presentation layer and creating dynamic web pages.

- A **Servlet** is written completely in Java code, which makes it more suitable for processing data, while a **JSP** contains HTML along with embedded Java code, making it easier to design user interfaces.
- A **Servlet** is compiled directly into bytecode and executed by the server, whereas a **JSP** is first converted into a Servlet and then compiled before execution.
- A **Servlet** is generally faster during the first execution since it does not require any conversion, while a **JSP** may take slightly more time initially due to the conversion step.
- In a **Servlet**, writing HTML code is difficult and time-consuming, whereas in a **JSP**, designing and managing HTML is much easier.
- A **Servlet** does not provide implicit objects directly and requires manual coding, while a **JSP** provides built-in implicit objects like request, response, and session.
- A **Servlet** is mainly used as a controller to handle application logic, whereas a **JSP** is used as a view to display the output to the user.

Q. Explain Action element in JSP and mention all the element in it and also state's it's use :

JSP action elements are special XML-like tags used in JSP to perform certain operations at runtime. These elements help in controlling the behavior of the JSP page and allow interaction with JavaBeans, other resources, and client requests. They are written using the <jsp:...> syntax and are processed on the server side when the page is execute.

Use of JSP Action Elements

JSP action elements are mainly used to simplify JSP development by reducing Java code inside HTML. They help in managing JavaBeans, reusing components, passing data between pages, and controlling page

flow. Overall, they make the web application more structured, maintainable, and easier to understand.

1. Forwarding (<jsp:forward>)

It is used to send the request from one JSP page to another page. The control is completely transferred and the current page stops executing.

2. Include (<jsp:include>)

It is used to include another page's content into the current JSP page at runtime. It helps in reusing common parts like header or footer.

3. Passing Parameters (<jsp:param>)

It is used to pass data (name and value) from one JSP page to another while forwarding or including.

4. Setting Properties (<jsp:setProperty>)

It is used to set or assign values to the properties of a JavaBean.

5. Naming Convention (JavaBean Rules)

JavaBean should have private variables, public getter and setter methods, and a default constructor.

Q. Explain the implicit object in JSP and state its use of it :

Implicit objects in JSP are predefined objects that are automatically created by the JSP container. The developer can directly use them in the

JSP page without explicitly creating them. These objects help in handling requests, responses, sessions, and other operations easily.

1. Request Object (Client Request Handling)

The **request** object is used to get information sent by the client to the server such as form data, headers, and cookies.

Important Methods

- **getParameter()** – Returns the value of a form field or query parameter.
- **getCookies()** – Returns all cookies sent by the client.
- **getContentLength()** – Returns the size of request data.

👉 **Use:** It is used to read user input and client request details.

2. Response Object (Server Response Handling)

The **response** object is used to send data from the server to the client and control the response.

Important Methods

- **addCookie()** – Adds a cookie to the response.
- **getContentType()** – Returns the type of response (like text/html).
- **getHeader()** – Returns response header information.

👉 **Use:** It is used to send output, set cookies, and control response behavior.

3. Session Object

The **session** object is used to store user data across multiple requests. It helps in maintaining user state (like login details).

Important Methods

- **setAttribute()** – Stores data in session.
- **getAttribute()** – Retrieves stored data.
- **invalidate()** – Ends the session.

👉 **Use:** It is used for session tracking and storing user-specific data.

4. Config Object

The **config** object contains configuration information of the servlet or JSP.

Important Methods

- **getInitParameter()** – Returns initialization parameters.
- **getServletName()** – Returns servlet name.

👉 **Use:** It is used to access configuration settings defined in web.xml.

Q. What is JSTL ? How JSTL fixes JSP scriplet's shortcomings and also state it's disadvantages :

JSTL (JavaServer Pages Standard Tag Library) is a collection of predefined tags used in JSP to perform common tasks like iteration, condition checking, formatting, and database access without writing Java code inside JSP pages.

Disadvantages of JSTL

- It requires proper configuration and library setup.
- It may be difficult for beginners to understand all tags initially.

- It is less flexible compared to writing full Java code in some complex cases.
- Performance may be slightly affected due to tag processing.

Earlier, JSP pages used **scriptlets (Java code inside JSP)** which made the code complex, hard to read, and difficult to maintain.

JSTL solves these problems in the following ways:

- It removes the need to write Java code inside JSP, making pages cleaner.
- It improves readability by using simple tag-based syntax.
- It separates business logic from presentation, following good design practices.
- It reduces errors and makes the code easier to maintain.

Q. Explain the Tag libraries of JSTL :

JSTL (JavaServer Pages Standard Tag Library) provides different tag libraries, each designed for a specific type of task. These libraries help in reducing Java code inside JSP and make development easier.

1. Core Tag Library

The core library is the most commonly used JSTL library. It provides tags for basic operations like iteration, condition checking, and URL handling.

👉 **Examples:** `<c:if>`, `<c:forEach>`, `<c:choose>`

👉 **Use:** Used for control flow (loops, conditions).

2. Formatting Tag Library

This library is used for formatting and internationalization (i18n). It helps in formatting numbers, dates, and handling different locales.

👉 **Examples:** `<fmt:formatNumber>`, `<fmt:formatDate>`

👉 **Use:** Used for formatting data and supporting multiple languages.

3. SQL Tag Library

This library provides tags to interact with databases directly from JSP.

👉 **Examples:** `<sql:query>`, `<sql:update>`

👉 **Use:** Used to perform database operations.

⚠️ *Note:* Not recommended in real projects (better to use DAO/Servlet).

4. XML Tag Library

This library is used to process and manipulate XML data.

👉 **Examples:** `<x:parse>`, `<x:forEach>`

👉 **Use:** Used when working with XML documents.

5. Functions Tag Library

This library provides functions to perform operations on strings and collections.

👉 **Examples:** `fn:length()`, `fn:contains()`

👉 **Use:** Used for string manipulation and utility functions.

UNIT – 4

Q. What is Enterprise java bean ? Explain it's architecture ?

Enterprise Java Bean (EJB) is a server-side component in Java used to build large-scale, secure, and distributed enterprise applications. It runs inside an EJB container and helps developers handle business logic such as transactions, security, and database operations automatically without writing complex code.

Architecture of Enterprise java bean are as follow :

1. EJB Server (Application Server)

EJB Server is the environment where the EJB container runs. It provides the platform and resources required to execute enterprise applications.

 **Use:** Used to host and run EJB components (example: GlassFish, JBoss).

2. EJB Container

EJB Container is a part of the application server that manages the execution of enterprise beans. It provides services like security, transaction management, lifecycle management, and concurrency.

 **Use:** It acts as a middle layer that handles all system-level services automatically.

3. Enterprise Bean (EJB)

Enterprise Bean is the main component that contains the business logic of the application. It performs operations like processing data, handling transactions, and interacting with the database.

👉 **Use:** Used to implement business functionality of the application.

4. Clients

Clients are the users or applications that access the enterprise beans. They can be web applications, desktop applications, or other enterprise components.

👉 **Use:** Used to send requests to EJB and receive responses.

Q. Explain the Types of Enterprise Bean :

Types of Enterprise Java Beans (EJB)

Enterprise Beans are mainly of two types: **Session Beans** and **Entity Beans**.

1. Session Bean

Session Beans are used to perform business logic for a client. They do not represent persistent data and exist only for the duration of the client session.

a) Stateless Session Bean

A Stateless Session Bean does not store any client-specific data. Each request is independent, and the bean can serve multiple clients.

👉 **Example:** Calculating total amount in an e-commerce app

👉 **Use:** Used when no need to maintain client state

b) Stateful Session Bean

A Stateful Session Bean stores data specific to a client across multiple requests. It maintains the state of interaction.

👉 **Example:** Shopping cart of a user

👉 **Use:** Used when client data needs to be remembered

2. Entity Bean

Entity Beans represent persistent data stored in a database. They are used to map database records into Java objects.

a) Bean-Managed Persistence (BMP)

In this type, the developer manually writes code to manage database operations like insert, update, and delete.

👉 **Use:** Gives full control over database operations

b) Container-Managed Persistence (CMP)

In this type, the EJB container automatically manages database operations. The developer does not need to write SQL code.

👉 **Use:** Simplifies development by reducing database handling code

Q. What is session beans ? Explain it's types :

A Session Bean is a type of Enterprise Java Bean (EJB) that is used to perform business logic for a client. It works on behalf of the client and does not represent persistent data. Session beans exist only for the duration of a client's interaction with the application.

Types of Session Beans

1. Stateless Session Bean

A Stateless Session Bean does not store any client-specific data. Each request is treated as independent, and the bean can be shared among multiple clients.

👉 **Example:** Login validation, calculations

👉 **Use:** Used when no need to maintain state between requests

2. Stateful Session Bean

A Stateful Session Bean stores client-specific data and maintains the state across multiple requests or interactions.

👉 **Example:** Shopping cart, user session data

👉 **Use:** Used when the application needs to remember client data

3. Singleton Session Bean

A Singleton Session Bean has only one instance for the entire application, and it is shared by all clients. It is used for shared resources or common data.

👉 **Example:** Application configuration, caching

👉 **Use:** Used when a single shared instance is required

Q. Explain the remote and local interface :

In Enterprise Java Beans (EJB), interfaces are used to define how clients interact with the bean. There are mainly two types of interfaces: **Remote Interface** and **Local Interface**.

1. Remote Interface

A Remote Interface is used when the client and the EJB are located on different machines or different JVMs. It allows communication over a network.

Features:

- Supports distributed applications
- Uses network communication (RMI)
- Slower compared to local due to network overhead

 **Use:** Used when the client is outside the server or in a different system.

2. Local Interface

A Local Interface is used when the client and the EJB are in the same application or same JVM. It allows direct method calls without network communication.

Features:

- Faster performance (no network overhead)
- Used within the same application
- More efficient for internal communication

 **Use:** Used when the client and bean are on the same server.

Q. What is message driven beans ? Explain how it works :

A Message-Driven Bean is a type of Enterprise Java Bean (EJB) that is used to process messages asynchronously. It does not interact directly with

clients; instead, it listens to messages sent to a messaging system (like a queue or topic) and processes them automatically.

How it Works

1. A client sends a message to a messaging system (queue/topic).
2. The EJB container listens for incoming messages.
3. When a message arrives, the container creates or assigns a Message-Driven Bean.
4. The bean processes the message using a method (like `onMessage()`).
5. After processing, the response or result is handled internally.

Life Cycle of Message-Driven Bean:

1. Creation

The EJB container creates instances of the Message-Driven Bean.

2. Initialization

The container initializes the bean and prepares it to receive messages.

3. Message Processing

When a message arrives, the container calls the `onMessage()` method of the bean to process it.

4. Removal

The container removes the bean instance when it is no longer needed.

Q. What is Interceptors and explain request, defining and invoke method ?

An Interceptor is a special mechanism in EJB used to intercept or capture method calls before or after the actual business method is executed. It is mainly used to add common services like logging, security, and transaction handling without modifying the main business logic.

Request Intercepting (Interception)

When a client sends a request to a bean method, the interceptor first captures this request. It can perform some processing (like logging or validation) and then pass the request to the actual method.

👉 *Simple meaning:* It acts like a filter between client and business method.

Defining an Interceptor

An interceptor is defined using a separate class and annotated with `@Interceptor`. The method inside it is marked with `@AroundInvoke`.

👉 *Simple meaning:* It is a class that contains logic to run before/after the main method.

Invoke Method (`@AroundInvoke`)

The interceptor method uses `@AroundInvoke` annotation. This method is called automatically when a business method is invoked.

- It receives an `InvocationContext` object
- It can execute code before and after the method
- It calls `context.proceed()` to continue execution

👉 *Simple meaning:* It controls the flow of method execution.

Q. Explain Java Naming and Directory Interface :

Java Naming and Directory Interface (JNDI) is an API used to access naming and directory services in Java. It allows applications to look up resources such as databases, EJBs, and other services using a name instead of hardcoding their details.

Q. Explain datasource resource definition in Java EE ?

DataSource Resource Definition in Java EE is a way to define a database connection resource directly inside the Java application using annotations. It allows the application to declare the database configuration without depending only on external server configuration.

How it Works (Simple)

1. Developer defines DataSource using annotation
2. Server reads and creates the resource
3. Application uses JNDI to access it
4. Connection is obtained and used

Use :

It is used to simplify database configuration and make the application self-contained, portable, and easy to deploy.

Key Points

- It is defined using annotation (`@DataSourceDefinition`)
- It reduces the need for server-side configuration
- It makes application more portable
- It still uses JNDI for lookup

UNIT – 5

Q. What is Persistence? Explain java persistence API ?

Persistence means storing data permanently so that it can be used later even after the program is closed. In Java, persistence is used to save data into a database.

Java Persistence API (JPA)

Java Persistence API (JPA) is a specification in Java used to manage and store data in databases using objects. It helps developers map Java classes to database tables and perform operations like insert, update, delete, and retrieve data easily.

What is Persistence?

Persistence is the process of storing data permanently so that it can be accessed and used later, even after the program has stopped running.

What is Persistence in Java?

Persistence in Java refers to storing Java object data into a permanent storage like a database or file, so that the data is not lost when the application ends. Java uses technologies like JDBC and JPA to achieve persistence.

👉 *Simple meaning:*

It is the process of **saving Java objects into a database and retrieving them when needed.**

What is ORM?

ORM stands for **Object Relational Mapping**. It is a technique used to connect **object-oriented programs** with **relational databases**.

ORM helps a programmer work with **Java objects** instead of writing too many SQL queries directly. It creates a mapping between the **class in Java** and the **table in the database**. In the same way, the **object** of the class represents a **row** of the table, and the **attributes or fields** of the class represent the **columns** of the table.

Why ORM is needed

In normal database programming, the developer has to manually write SQL queries for inserting, updating, deleting, and fetching data. This takes more time and makes the code lengthy. ORM solves this problem by automatically handling the conversion between objects and database tables. Because of this, development becomes easier, faster, and more organized.

How ORM works

ORM works by creating a relationship between:

- **Class and Table**
- **Object and Row**
- **Fields and Columns**

When the programmer performs operations on objects, ORM internally converts them into SQL operations.

For example:

- creating an object and saving it → **INSERT**
- modifying an object → **UPDATE**
- deleting an object → **DELETE**
- reading an object → **SELECT**

Thus, ORM acts like a bridge between the Java application and the database.

Benefits of ORM in Software Development

- Reduces the need to write complex SQL queries
- Saves development time and increases productivity
- Improves code readability and maintainability
- Allows developers to work with objects instead of database tables
- Provides database independence (portable code)
- Automatically handles data conversion between objects and tables

What is Java Persistence API (JPA)?

Java Persistence API (JPA) is a Java specification used to manage and store data in a database using Java objects. It helps in mapping Java classes to database tables and performing operations like insert, update, delete, and retrieve without writing complex SQL queries.

Architecture of JPA (Simple Explanation)

JPA architecture shows how Java application interacts with the database using different components inside the **javax.persistence** package.

Main Components

1. Entity

Entity is a simple Java class that represents a table in the database. Each object of the entity represents one row in the table.

👉 *Simple: Class = Table, Object = Row*

2. Entity Manager

Entity Manager is the main interface that performs operations like insert, update, delete, and fetch data.

👉 *Simple:* It controls all database operations.

3. Entity Manager Factory

It is used to create multiple EntityManager objects.

👉 *Simple:* Factory that gives EntityManager.

4. Entity Transaction

It manages transactions like begin, commit, and rollback.

👉 *Simple:* Ensures data is saved properly and safely.

5. Persistence

This class is used to get the EntityManagerFactory object.

👉 *Simple:* Starting point of JPA.

6. Query

Used to write and execute queries to fetch data from the database.

👉 *Simple:* Used to get specific data.

7. JPA API

It connects all these components and interacts with the database.

Q. Explain what is Hibernate ?

Hibernate is an open-source ORM (Object Relational Mapping) framework in Java used to simplify database operations. It allows developers to interact with the database using Java objects instead of writing complex SQL queries.

Features / Advantages of Hibernate

1. ORM Support

Hibernate maps Java classes to database tables and handles data conversion automatically.

2. Reduces SQL Code

Developers do not need to write complex SQL queries; Hibernate generates them automatically.

3. Database Independence

Hibernate supports multiple databases (MySQL, Oracle, etc.) without changing much code.

4. Automatic Table Creation

It can automatically create and update database tables based on Java classes.

5. Caching Mechanism

Hibernate provides first-level and second-level caching, which improves performance.

6. Transaction Management

It supports transaction handling to ensure data consistency.

7. HQL (Hibernate Query Language)

Hibernate provides HQL, which is similar to SQL but works with objects instead of tables.

8. Easy Integration

It can be easily integrated with frameworks like Spring.

Architecture of Hibernate

Hibernate architecture shows how all components work together to interact with the database.

1. Configuration

This is used to configure Hibernate. It contains database details like URL, username, password, and mapping information.

👉 *Simple:* It sets up Hibernate.

2. SessionFactory

It is a factory that creates Session objects. It is created once and used throughout the application.

👉 *Simple:* Factory to create sessions.

3. Session

Session is the main interface used to interact with the database. It performs operations like insert, update, delete, and fetch.

👉 *Simple:* Works like EntityManager in JPA.

4. Transaction

It is used to manage database transactions (begin, commit, rollback).

👉 *Simple:* Ensures safe and consistent data operations.

5. Query

Used to retrieve data from the database using HQL (Hibernate Query Language).

👉 *Simple:* Used to fetch specific data.

6. Criteria (Optional)

Used to create queries in a programmatic way instead of writing HQL.

👉 *Simple:* Query without writing SQL/HQL.

Components of Hibernate (JTA, JDBC, JNDI)

Hibernate uses some external technologies to perform its operations smoothly. The main components are:

1. JDBC (Java Database Connectivity)

JDBC is used by Hibernate to communicate with the database. It helps in executing SQL queries and managing database connections.

👉 *Simple meaning:*

Hibernate internally uses JDBC to **connect and talk to the database**.

👉 **Use:**

Used to perform actual database operations like insert, update, delete, and select.

2. JTA (Java Transaction API)

JTA is used to manage transactions in Hibernate, especially in enterprise applications. It ensures that all operations in a transaction are completed successfully or rolled back in case of failure.

👉 *Simple meaning:*

It manages **transaction control (commit/rollback)**.

👉 **Use:**

Used to maintain data consistency and handle multiple operations as a single unit.

3. JNDI (Java Naming and Directory Interface)

JNDI is used to look up resources like DataSource (database connection) by name instead of hardcoding details.

👉 *Simple meaning:*

It helps Hibernate **find database connection using a name**.

👉 Use:

Used to access configured resources (like database connection) from the server.

Q. Impedance Mismatch?

Impedance Mismatch:

Impedance mismatch is a problem that occurs when there is a difference between the way data is represented in an **object-oriented programming language (like Java)** and the way data is stored in a **relational database**.

Explanation

In Java, data is represented using objects, classes, inheritance, and relationships.

In databases, data is stored in tables with rows and columns.

Because of this difference:

- Java uses objects and references
- Database uses tables and foreign keys

This mismatch makes it complex to convert data between objects and tables.

Q. explain what is JPQL ?

What is JPQL?

JPQL (Java Persistence Query Language) is a query language used in JPA to retrieve and manipulate data from the database. It is similar to SQL but works with **Java objects (entities)** instead of database tables.

Explanation

In SQL, we write queries using table names and column names.

But in JPQL, we write queries using **entity class names and their fields**.

👉 *Example:*

- SQL → works on tables
 - JPQL → works on Java classes (entities)
-

Key Features

- Object-oriented (uses classes and fields)
- Database independent
- Supports SELECT, UPDATE, DELETE operations
- Works with EntityManager

Use

JPQL is used to fetch and manipulate data in JPA applications in a simpler and object-oriented way without directly writing SQL queries.

Q. explain the creation of config file in hibernate

Creation of Configuration File in Hibernate

In Hibernate, a configuration file is used to define database connection details and mapping information. The most commonly used configuration file is **hibernate.cfg.xml**.

Steps to Create Configuration File

1. Create XML File

Create a file named **hibernate.cfg.xml** in the project (usually inside src folder).

2. Add Hibernate Configuration Tags

The file starts with standard XML and Hibernate configuration tags.

👉 It contains `<hibernate-configuration>` and `<session-factory>`.

3. Provide Database Details

Inside the file, specify:

- Database URL
 - Username and Password
 - Driver class
 - Dialect (database type like MySQL, Oracle)
-

4. Add Mapping Information

Mention the mapping of Java classes (entity classes) to database tables using mapping tags.

5. Save and Use in Application

Hibernate reads this file at runtime to establish connection and manage database operations.

Simple Example Structure (Conceptual)

- Configuration root
- Session factory
- Database properties

- Mapping of classes
-

Use

The configuration file is used to connect Hibernate with the database and define how Java classes are mapped to database tables.

